Lab Digital Assignment - 1

Course Name : Agile Development Process and Devops Lab

Course code : ISWE406P

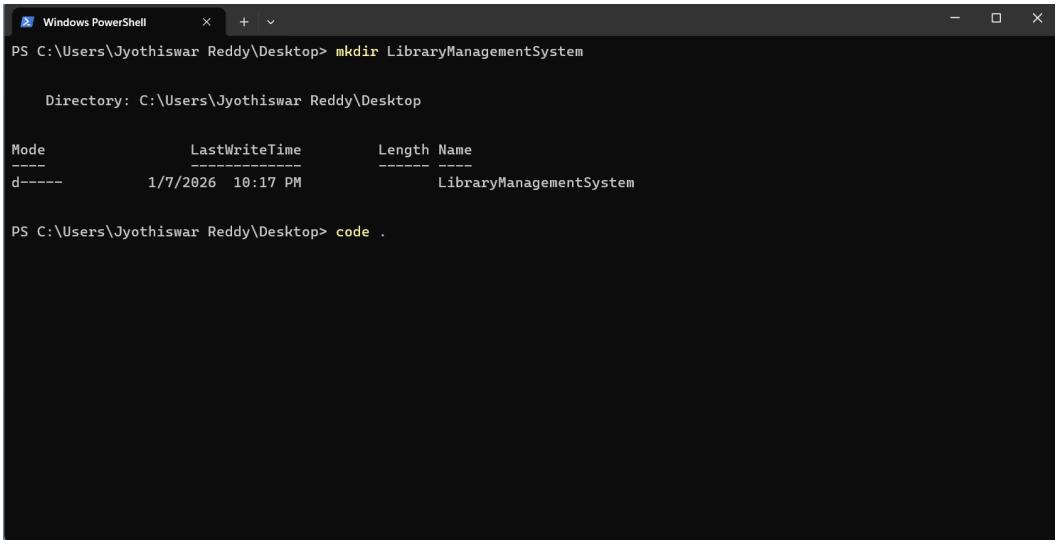
Name : CP . Jyotheswar Reddy

Reg No : 23MIS0097

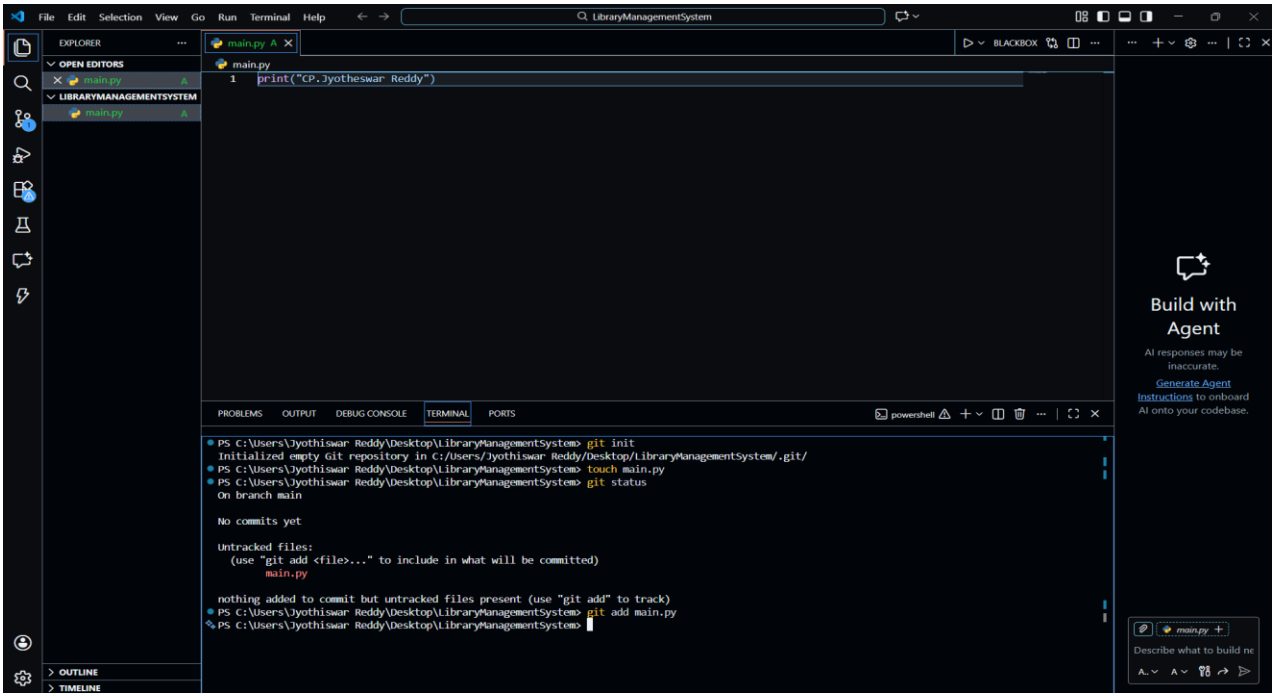
Case Study 1:

New Project Version Control Setup Scenario: You are assigned to develop a Library Management System. Before coding, your team decides to enable version control.

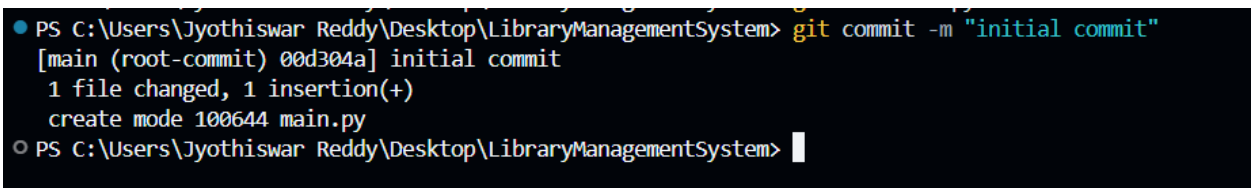
a) Create a project folder and open it in VS Code



b) Initialize a Git repository & Create a Python file and track it using Git



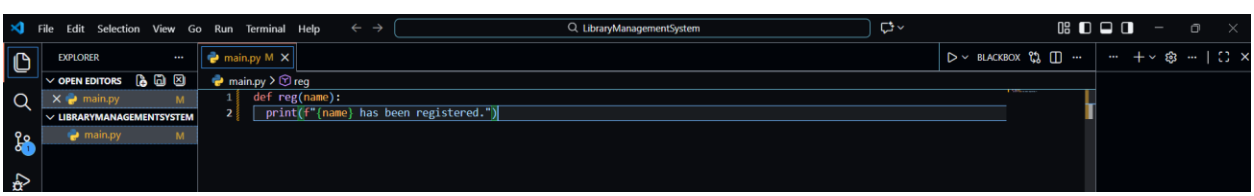
c) Commit the initial version



Case Study 2:

Tracking Daily Code Changes Scenario: You are working on a Student Registration Module. You make small updates daily and need to record them.

a) Modify an existing file



b) Check file differences

```
 create mode 100644 main.py
PS C:\Users\Jyothiswar Reddy\Desktop\LibraryManagementSystem> git diff
diff --git a/main.py b/main.py
index b6f0aca..73f0e40 100644
--- a/main.py
+++ b/main.py
@@ -1 +1,2 @@
-print("CP.Jyotheswar Reddy")
\ No newline at end of file
+def reg(name):
+  print(f"{name} has been registered.")
\ No newline at end of file
```

c) Stage and commit changes

```
PS C:\Users\Jyothiswar Reddy\Desktop\LibraryManagementSystem> git add main.py
PS C:\Users\Jyothiswar Reddy\Desktop\LibraryManagementSystem> git commit -m "second commit"
[main 726aba5] second commit
 1 file changed, 2 insertions(+), 1 deletion(-)
PS C:\Users\Jyothiswar Reddy\Desktop\LibraryManagementSystem>
```

d) View commit history

```
PS C:\Users\Jyothiswar Reddy\Desktop\LibraryManagementSystem> git log
commit 726aba5b535e6a2ffbf616f0eb522d544855a071 (HEAD -> main)
Author: Jyotheswar18 <Chinnapapireddygari2023@vitstudent.ac.in>
Date:   Wed Jan 7 22:43:00 2026 +0530

    second commit

commit 00d304aff3b71e34a22a21038b812066a6f3d574
Author: Jyotheswar18 <Chinnapapireddygari2023@vitstudent.ac.in>
Date:   Wed Jan 7 22:28:18 2026 +0530

    initial commit
PS C:\Users\Jyothiswar Reddy\Desktop\LibraryManagementSystem>
```

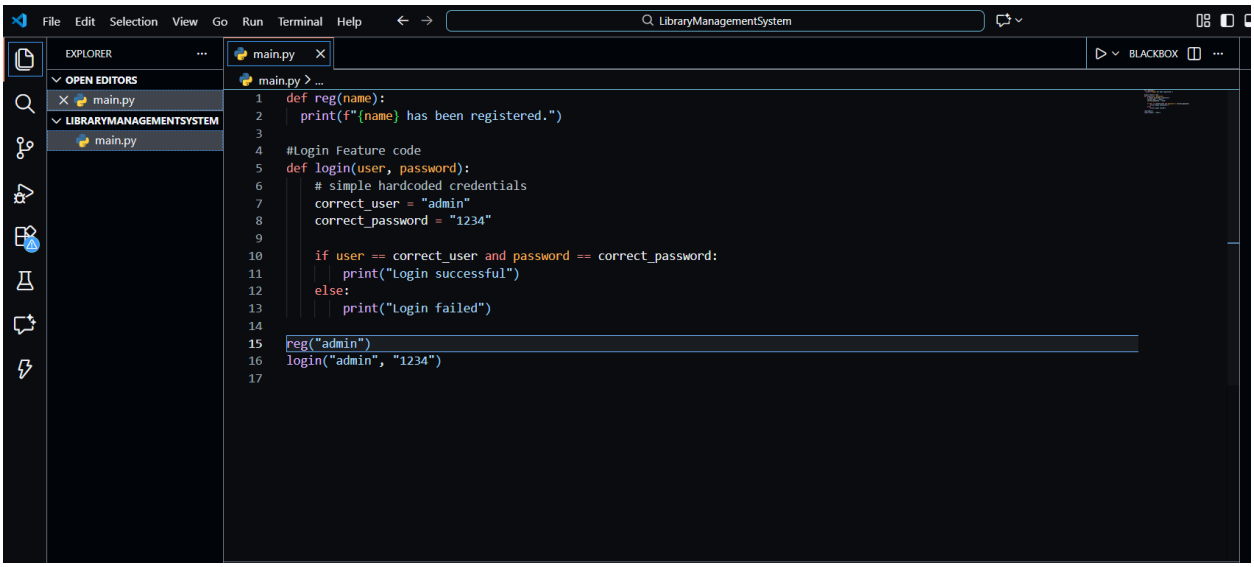
Case Study 3:

Feature Development Using Branches Scenario: A new Login feature must be developed without disturbing main code.

a) Create a new branch

```
PS C:\Users\Jyothiswar Reddy\Desktop\LibraryManagementSystem> git branch login-feature
PS C:\Users\Jyothiswar Reddy\Desktop\LibraryManagementSystem> git checkout login-feature
Switched to branch 'login-feature'
PS C:\Users\Jyothiswar Reddy\Desktop\LibraryManagementSystem>
```

b) Add login-related code

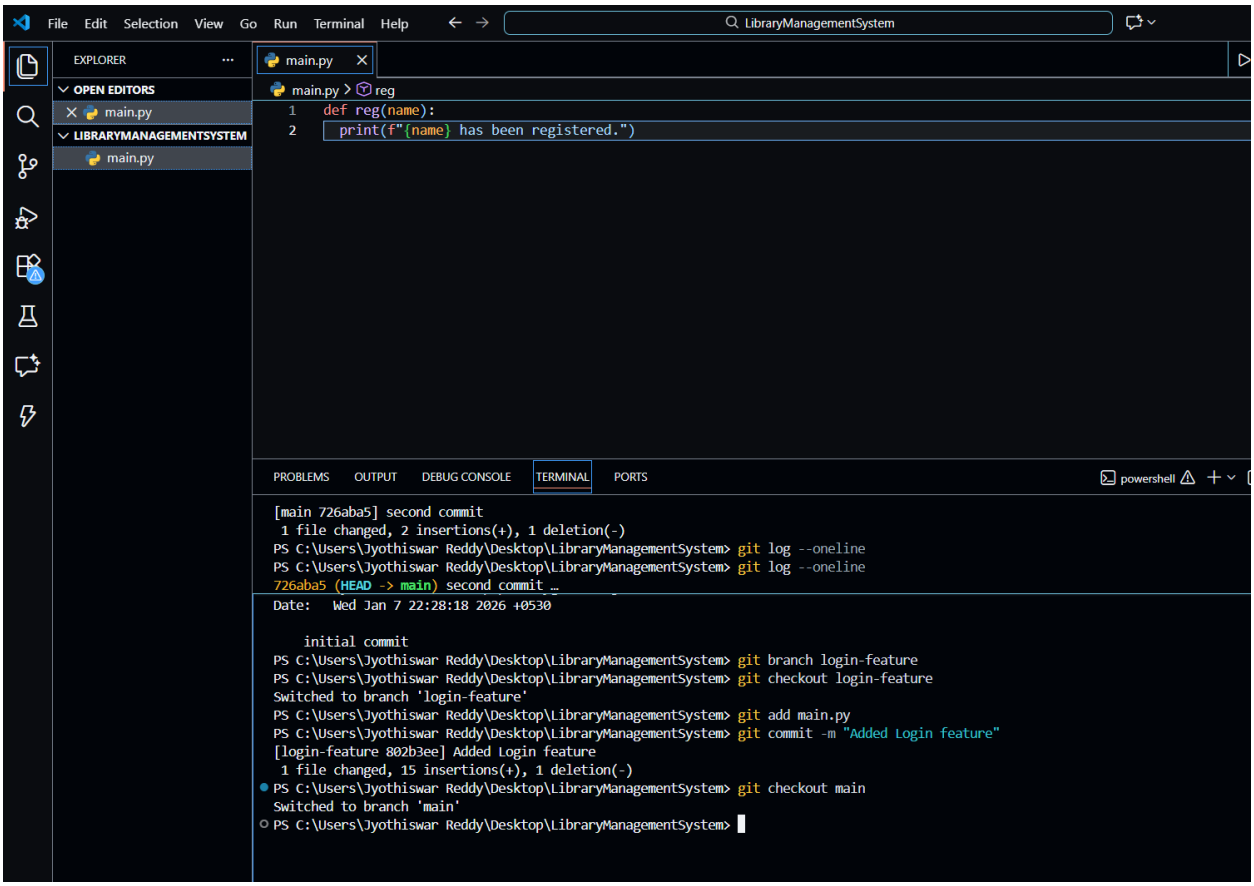


c) Commit changes

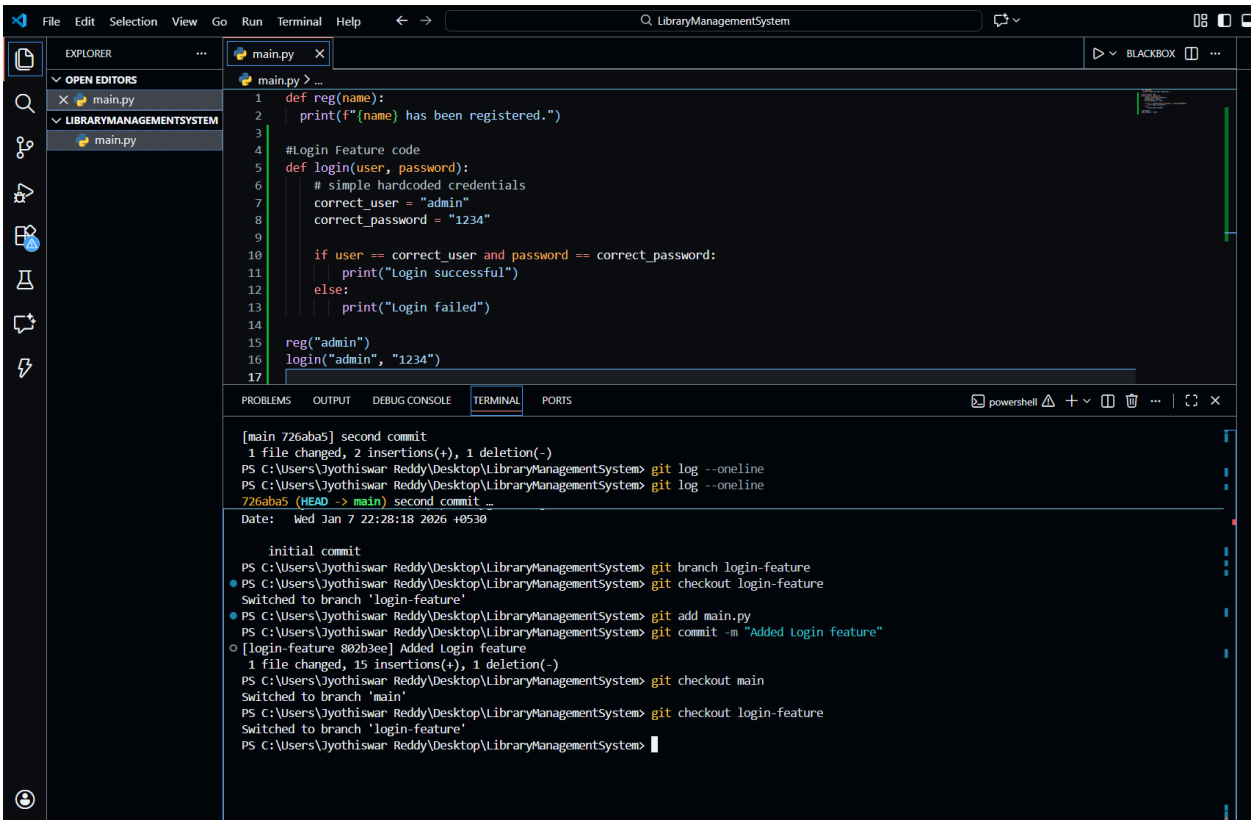
```
PS C:\Users\Jyothiswar Reddy\Desktop\LibraryManagementSystem> git add main.py
PS C:\Users\Jyothiswar Reddy\Desktop\LibraryManagementSystem> git commit -m "Added Login feature"
[login-feature 802b3ee] Added Login feature
 1 file changed, 15 insertions(+), 1 deletion(-)
PS C:\Users\Jyothiswar Reddy\Desktop\LibraryManagementSystem>
```

d) Switch between branches

i) Switched to main branch



ii) Switched to the login-feature branch



Case Study 4:

Merging Feature into Main Code Scenario: The login feature is tested and approved.

a) Switch to main branch

```
def reg(name):
    print(f"{name} has been registered.")
```

```
[main 726aba5] second commit
 1 file changed, 2 insertions(+), 1 deletion(-)
PS C:\Users\Jyothiswar Reddy\Desktop\LibraryManagementSystem> git log --oneline
PS C:\Users\Jyothiswar Reddy\Desktop\LibraryManagementSystem> git log --oneline
726aba5 (HEAD -> main) second commit …
Date:   Wed Jan 7 22:28:18 2026 +0530

    initial commit
PS C:\Users\Jyothiswar Reddy\Desktop\LibraryManagementSystem> git branch login-feature
PS C:\Users\Jyothiswar Reddy\Desktop\LibraryManagementSystem> git checkout login-feature
Switched to branch 'login-feature'
PS C:\Users\Jyothiswar Reddy\Desktop\LibraryManagementSystem> git add main.py
PS C:\Users\Jyothiswar Reddy\Desktop\LibraryManagementSystem> git commit -m "Added Login feature"
[login-feature 802b3ee] Added Login feature
 1 file changed, 15 insertions(+), 1 deletion(-)
PS C:\Users\Jyothiswar Reddy\Desktop\LibraryManagementSystem> git checkout main
Switched to branch 'main'
PS C:\Users\Jyothiswar Reddy\Desktop\LibraryManagementSystem>
```

b) Merge feature branch

```
def reg(name):
    print(f"{name} has been registered.")

#Login Feature code
def login(user, password):
    # simple hardcoded credentials
    correct_user = "admin"
    correct_password = "1234"

    if user == correct_user and password == correct_password:
        print("Login successful")
    else:
        print("Login failed")

reg("admin")
login("admin", "1234")
```

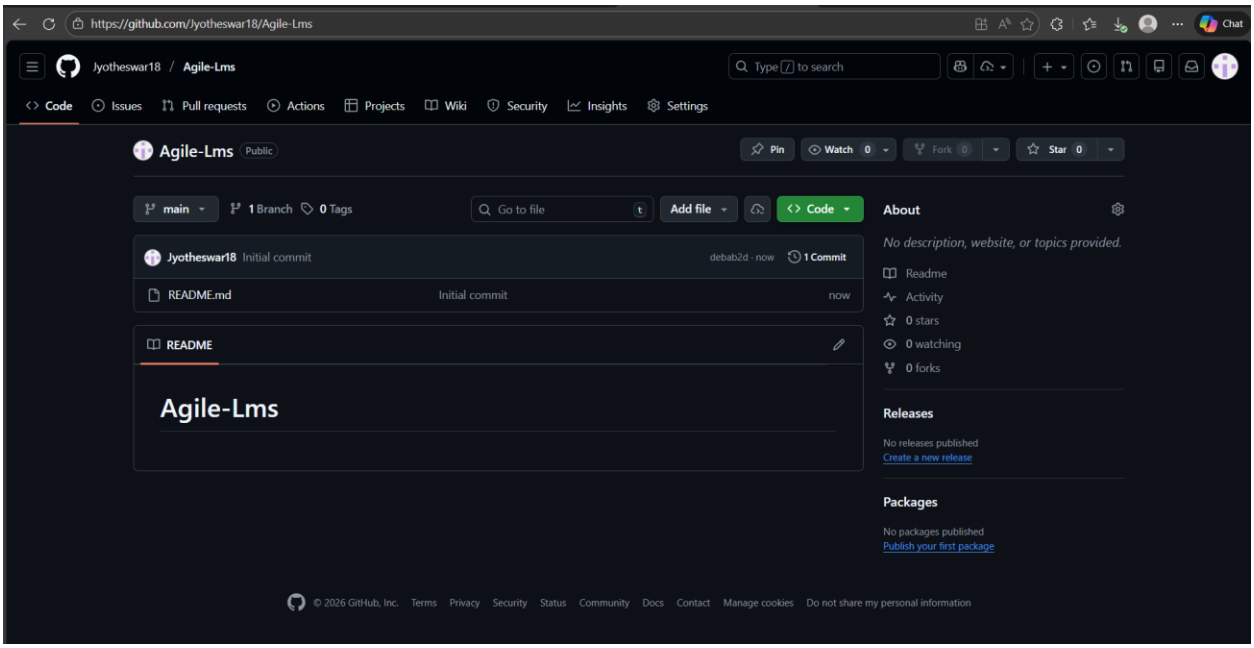
```
[main 726aba5] second commit
 1 file changed, 2 insertions(+), 1 deletion(-)
PS C:\Users\Jyothiswar Reddy\Desktop\LibraryManagementSystem> git log --oneline
PS C:\Users\Jyothiswar Reddy\Desktop\LibraryManagementSystem> git log --oneline
726aba5 (HEAD -> main) second commit …
    initial commit
PS C:\Users\Jyothiswar Reddy\Desktop\LibraryManagementSystem> git branch login-feature
PS C:\Users\Jyothiswar Reddy\Desktop\LibraryManagementSystem> git checkout login-feature
Switched to branch 'login-feature'
PS C:\Users\Jyothiswar Reddy\Desktop\LibraryManagementSystem> git add main.py
PS C:\Users\Jyothiswar Reddy\Desktop\LibraryManagementSystem> git commit -m "Added Login feature"
[login-feature 802b3ee] Added Login feature
 1 file changed, 15 insertions(+), 1 deletion(-)
PS C:\Users\Jyothiswar Reddy\Desktop\LibraryManagementSystem> git checkout main
Switched to branch 'main'
PS C:\Users\Jyothiswar Reddy\Desktop\LibraryManagementSystem> git checkout login-feature
Switched to branch 'login-feature'
PS C:\Users\Jyothiswar Reddy\Desktop\LibraryManagementSystem> git merge login-feature
Already up to date.
PS C:\Users\Jyothiswar Reddy\Desktop\LibraryManagementSystem> git checkout main
Switched to branch 'main'
PS C:\Users\Jyothiswar Reddy\Desktop\LibraryManagementSystem> git merge login-feature
Updating 726aba5..802b3ee
Fast-forward
 main.py | 16 ++++++++++++++-
 1 file changed, 15 insertions(+), 1 deletion(-)
PS C:\Users\Jyothiswar Reddy\Desktop\LibraryManagementSystem>
```

c) Verify merged content

Case Study 5:

Working with Remote Repository Scenario: The project must be shared using GitHub.

a) Create remote repository



b) Add remote origin & Push code to GitHub

```
def reg(name):
    print(f"{name} has been registered.")

#Login Feature code
def login(user, password):
    # simple hardcoded credentials
    correct_user = "admin"
    correct_password = "1234"
```

```
PS C:\Users\Jyothiswar Reddy\Desktop\LibraryManagementSystem> git remote add origin https://github.com/Jyotheswar18/Agile-Lms.git
>> git branch -M main
>> git push -u origin main
error: remote origin already exists.
Enumerating objects: 9, done.
Counting objects: 100% (9/9), done.
Delta compression using up to 12 threads
Compressing objects: 100% (5/5), done.
Writing objects: 100% (9/9), 883 bytes | 147.00 KiB/s, done.
Total 9 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), done.
To https://github.com/Jyotheswar18/Agile-Lms.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
PS C:\Users\Jyothiswar Reddy\Desktop\LibraryManagementSystem>
```

Case Study 6:

Undoing Mistakes Scenario: A file is modified incorrectly.

a)View file changes

```
PS C:\Users\Jyothiswar Reddy\Desktop\LibraryManagementSystem> git status
On branch main
Your branch is up to date with 'origin/main'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   main.py

no changes added to commit (use "git add" and/or "git commit -a")
PS C:\Users\Jyothiswar Reddy\Desktop\LibraryManagementSystem> git diff
     # simple hardcoded credentials
-    correct_user = "admin"
+    correct_user = "Jyotheswar"
+login("Jyotheswar", "1234")
+print("This is the main.py file.")
\ No newline at end of file
:...skipping...
diff --git a/main.py b/main.py
index f1a2796..7461ee1 100644
--- a/main.py
+++ b/main.py
@@ -4,7 +4,7 @@ def reg(name):
 #Login Feature code
 def login(user, password):
     # simple hardcoded credentials
-    correct_user = "admin"
+    correct_user = "Jyotheswar"
     else:
         print("Login failed")

-reg("admin")
-login("admin", "1234")
+reg("Jyotheswar")
+login("Jyotheswar", "1234")
+print("This is the main.py file.")
\ No newline at end of file
~
```

b) Discard uncommitted changes

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\Users\Jyothiswar Reddy\Desktop\LibraryManagementSystem> git checkout -- main.py
PS C:\Users\Jyothiswar Reddy\Desktop\LibraryManagementSystem>
```

Case Study 7:

Handling Merge Conflicts Scenario: Two developers edited the same file.

a)Merge branches

Showing 1 changed file with 4 additions and 4 deletions. Split Unified

main.py

```
@@ -1,16 +1,16 @@
 def register(name):
-    print(f"Student {name} registered successfully.")
+    print(f"{name} has been registered.")

 #Login Feature code
 def login(user, password):
+    # simple hardcoded credentials
     correct_user = "admin"
-    correct_password = "12345"
+    correct_password = "12345678"

     if user == correct_user and password == correct_password:
         print("Login successful")
     else:
-        print("Invalid username or password")
+        print("Login failed")

 register("admin")
 login("admin", "1234")
```

Login feature #2

Resolving conflicts between `login-feature` and `main` and committing changes → `login-feature`

1 conflicting file | main.py | 1 conflict | Prev | Next | Mark as resolved

main.py
main.py

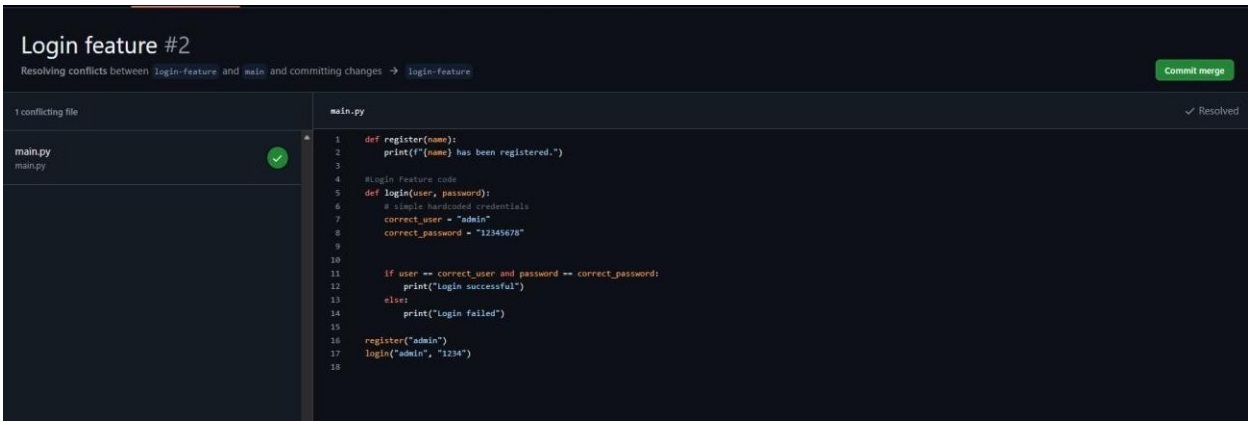
```
def register(name):
    print(f"{name} has been registered.")

#Login Feature code
def login(user, password):
Accept current change | Accept incoming change | Accept both changes
<<<<<<< login-feature (Current change)
    # simple hardcoded credentials
    correct_user = "admin"
    correct_password = "12345678"
=======
    correct_user = "admin1"
    correct_password = "12345"
>>>>>>> main (Incoming change)

    if user == correct_user and password == correct_password:
        print("login successful")
    else:
        print("login failed")

register("admin")
login("admin", "1234")
```

b)Resolve conflicts

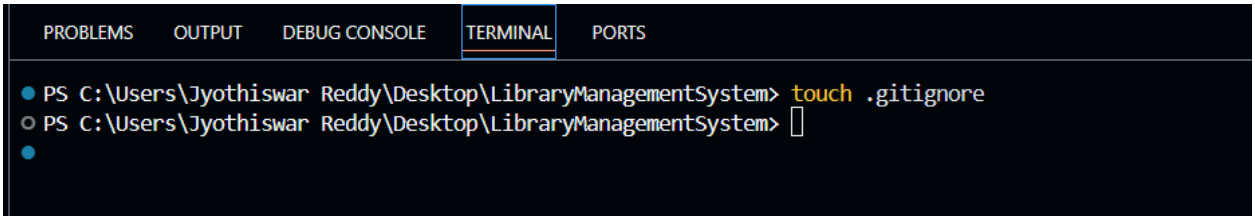


Case Study 8:

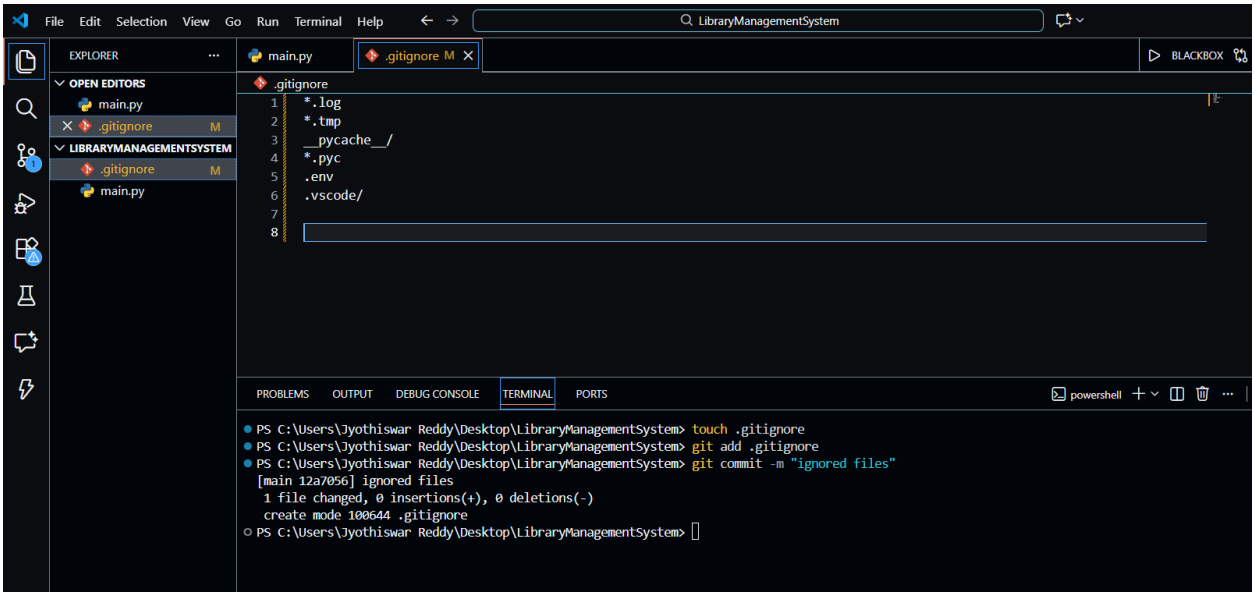
Ignoring Unwanted Files Scenario: Log and temporary files should not be tracked.

Tasks:

a) Create .gitignore file



b) Add ignore patterns



Case Study 9:

Collaborating with Team Scenario: A teammate pushed new changes.

a) Fetch latest changes

```
PS C:\Users\Jyothiswar Reddy\Desktop\LibraryManagementSystem> git fetch origin
PS C:\Users\Jyothiswar Reddy\Desktop\LibraryManagementSystem>
```

b)Pull updates

```
PS C:\Users\Jyothiswar Reddy\Desktop\LibraryManagementSystem> git pull origin main
From https://github.com/Jyotheswar18/Agile-Lms
 * branch            main       -> FETCH_HEAD
Already up to date.
PS C:\Users\Jyothiswar Reddy\Desktop\LibraryManagementSystem>
```

c)Resolve conflicts if any

No conflicts

Case Study 10:

Final Project Release Scenario: Project is ready for final submission.

a) Review commit history

```
PS C:\Users\Jyothiswar Reddy\Desktop\LibraryManagementSystem> git log --oneline
12a7056 (HEAD -> main) ignored files
802b3ee (origin/main, origin/HEAD, login-feature) Added Login feature
726aba5 second commit
00d304a initial commit
PS C:\Users\Jyothiswar Reddy\Desktop\LibraryManagementSystem>
```

b)Push final code

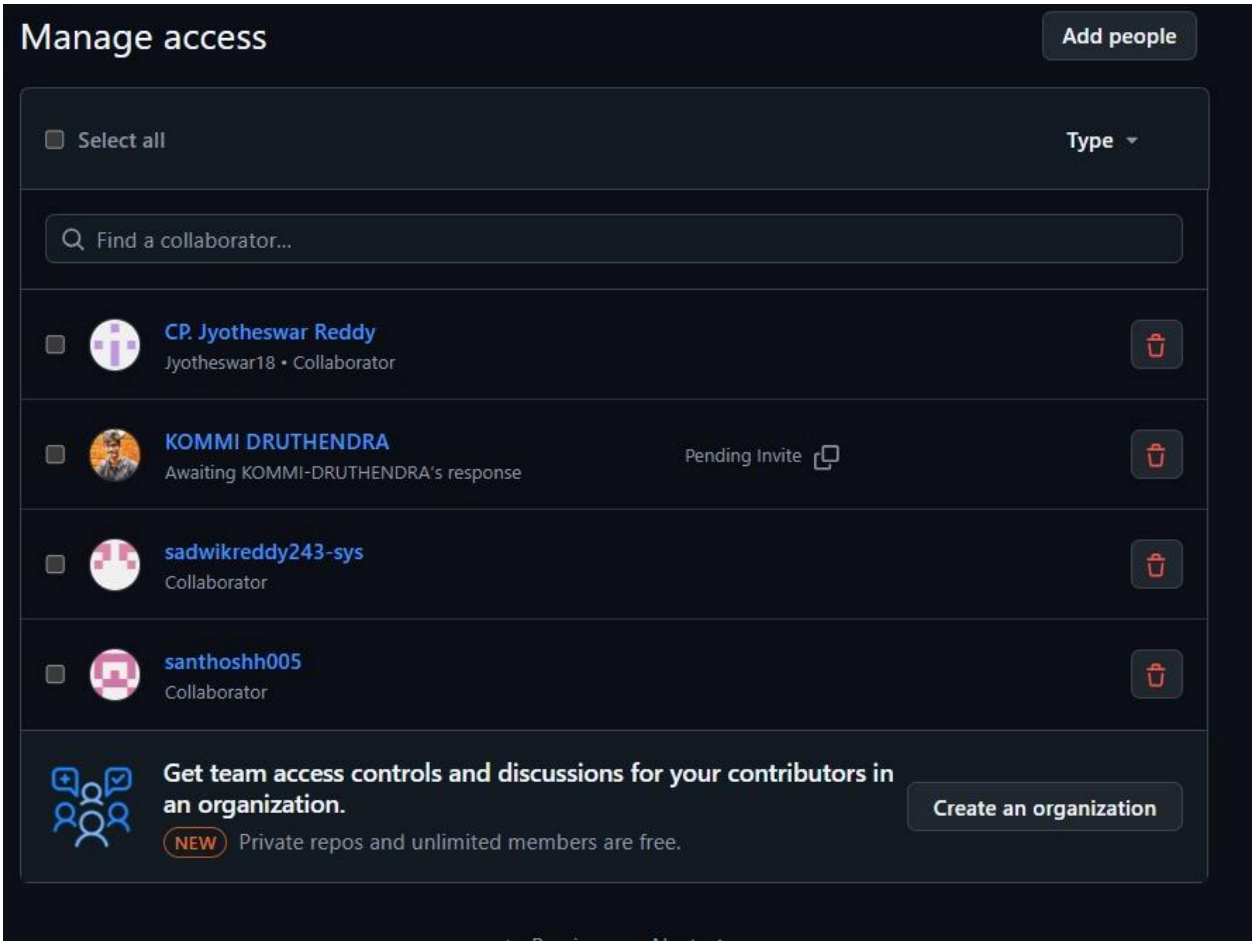
```
PS C:\Users\Jyothiswar Reddy\Desktop\LibraryManagementSystem> git push origin main
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 12 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 297 bytes | 297.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/Jyotheswar18/Agile-Lms.git
   802b3ee..12a7056  main -> main
PS C:\Users\Jyothiswar Reddy\Desktop\LibraryManagementSystem>
```

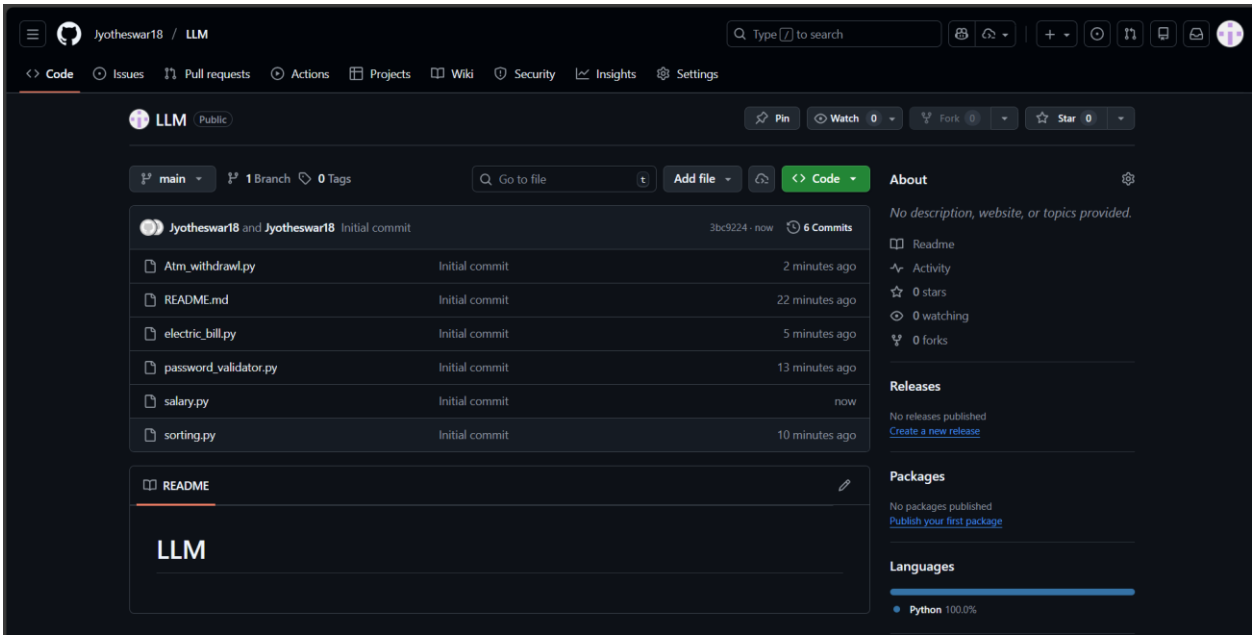
c)Prepare release version

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\Users\Jyothiswar Reddy\Desktop\LibraryManagementSystem> git tag v1.0
fatal: tag 'v1.0' already exists
PS C:\Users\Jyothiswar Reddy\Desktop\LibraryManagementSystem> git push origin v1.0
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/Jyotheswar18/Agile-Lms.git
 * [new tag]         v1.0 -> v1.0
PS C:\Users\Jyothiswar Reddy\Desktop\LibraryManagementSystem>
```

Class Assessment





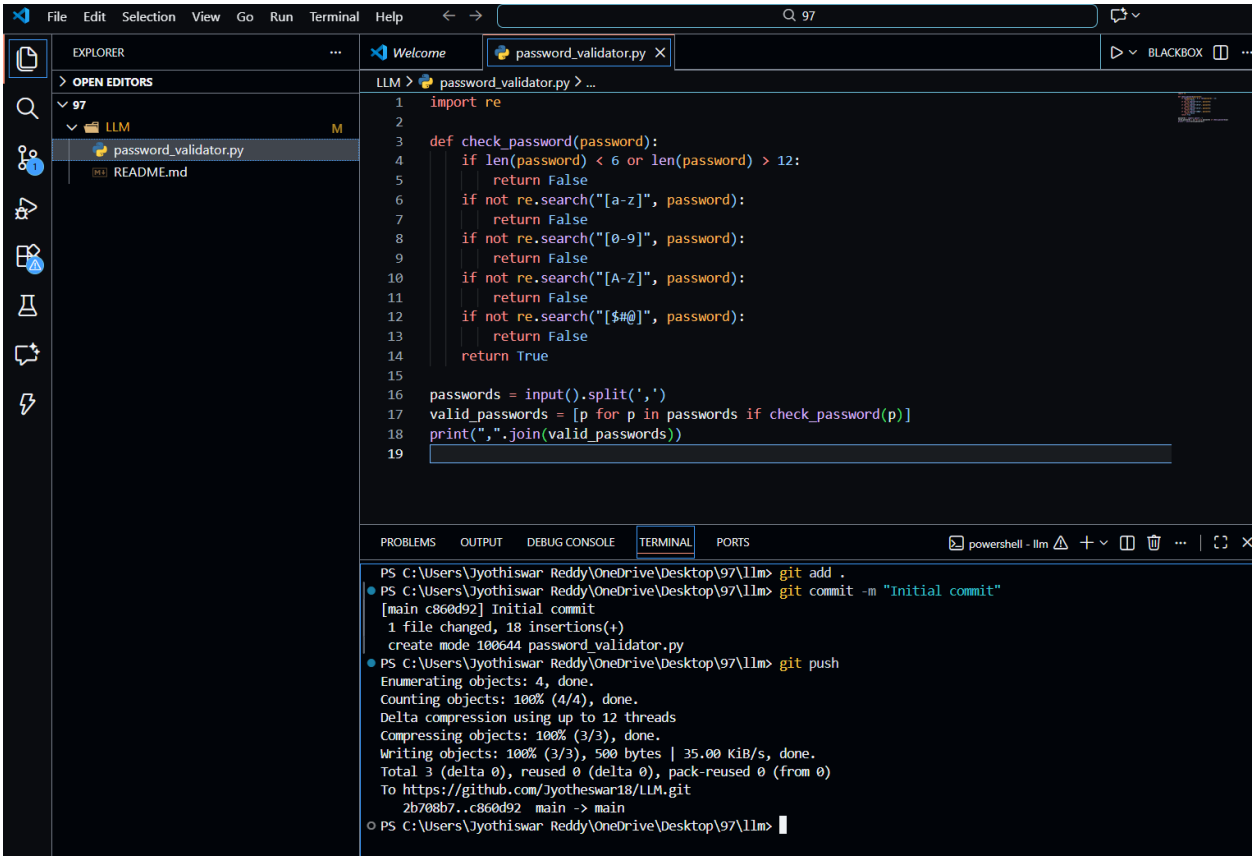
1) A website requires the users to input username and password to register. Write a program to check the validity of password input by users. Following are the criteria for checking the password:

1. At least 1 letter between [a-z]
2. At least 1 number between [0-9] 1. At least 1 letter between [A-Z]
3. At least 1 character from [$#@]
4. Minimum length of transaction password: 6
5. Maximum length of transaction password: 12 Your program should accept a sequence of comma separated passwords and will check them according to the above criteria. Passwords that match the criteria are to be printed, each separated by a comma.

Example If the following passwords are given as

input to the program: ABd1234@1,a F1#,2w3E*,2We3345

Then, the output of the program should be: ABd1234@1



2) You are required to write a program to sort the (name, age, height) tuples by ascending order where name is string, age and height are numbers. The tuples are input by console. The sort criteria is:

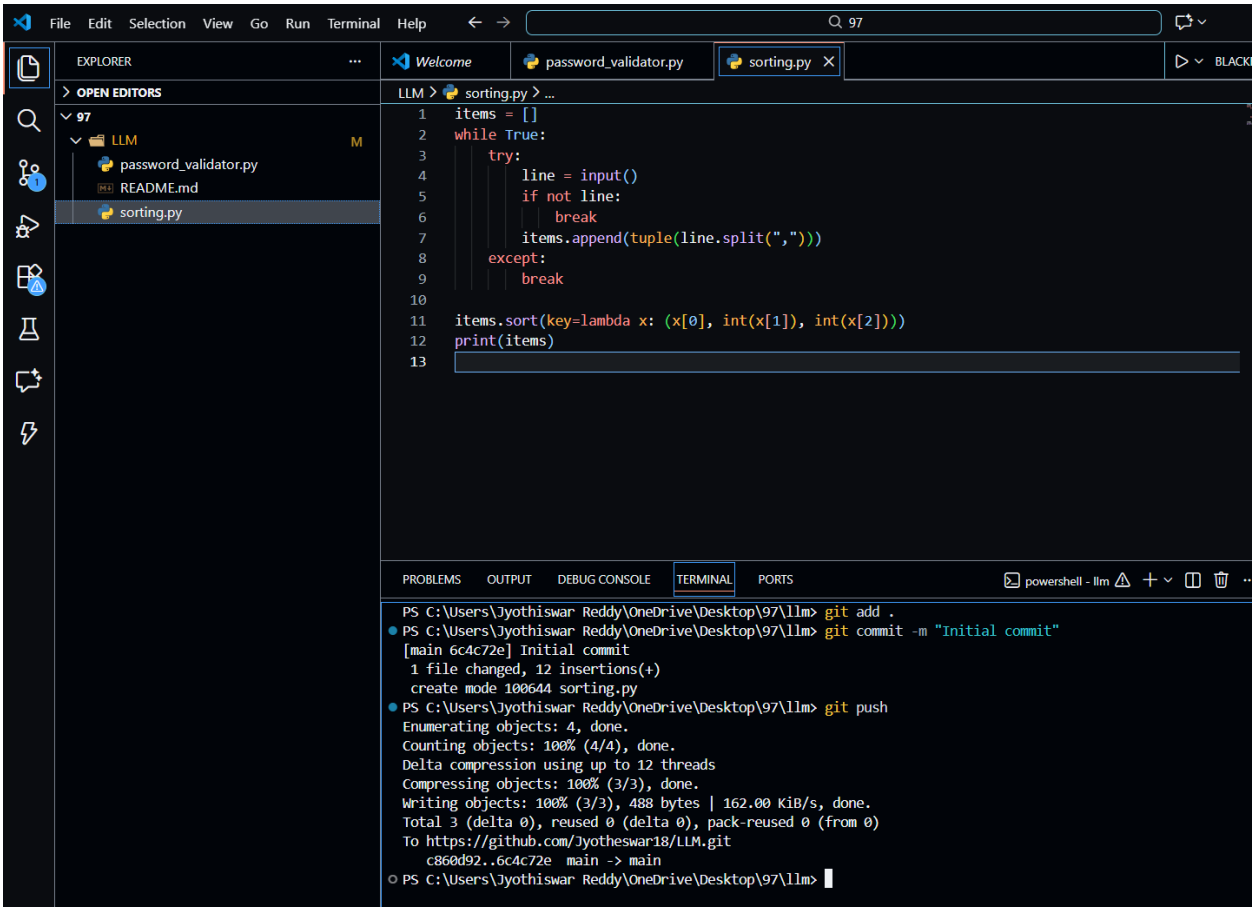
1: Sort based on name;

2: Then sort based on age;

3: Then sort by score. The priority is that name > age > score. If the following tuples are given as input to the program:

Tom,19,80 John,20,90 Jony,17,91 Jony,17,93 Json,21,85

Then, the output of the program should be: [('John', '20', '90'), ('Jony', '17', '91'), ('Jony', '17', '93'), ('Json', '21', '85'), ('Tom', '19', '80')]



3) An electricity board calculates charges based on usage. Write a program that: • Takes number of units consumed •

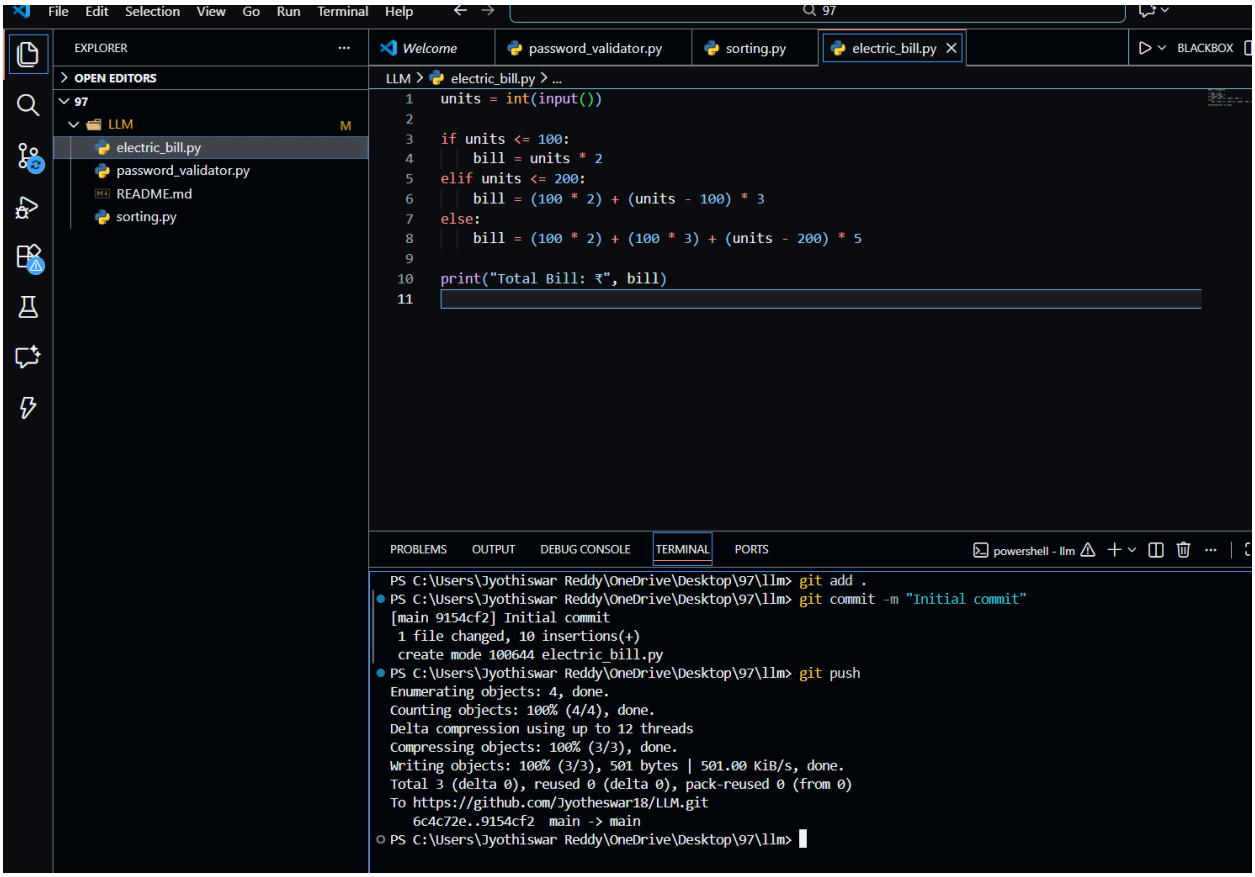
Calculates bill:

First 100 units → ₹2/unit

Next 100 units → ₹3/unit

Remaining → ₹5/unit

Displays total bill

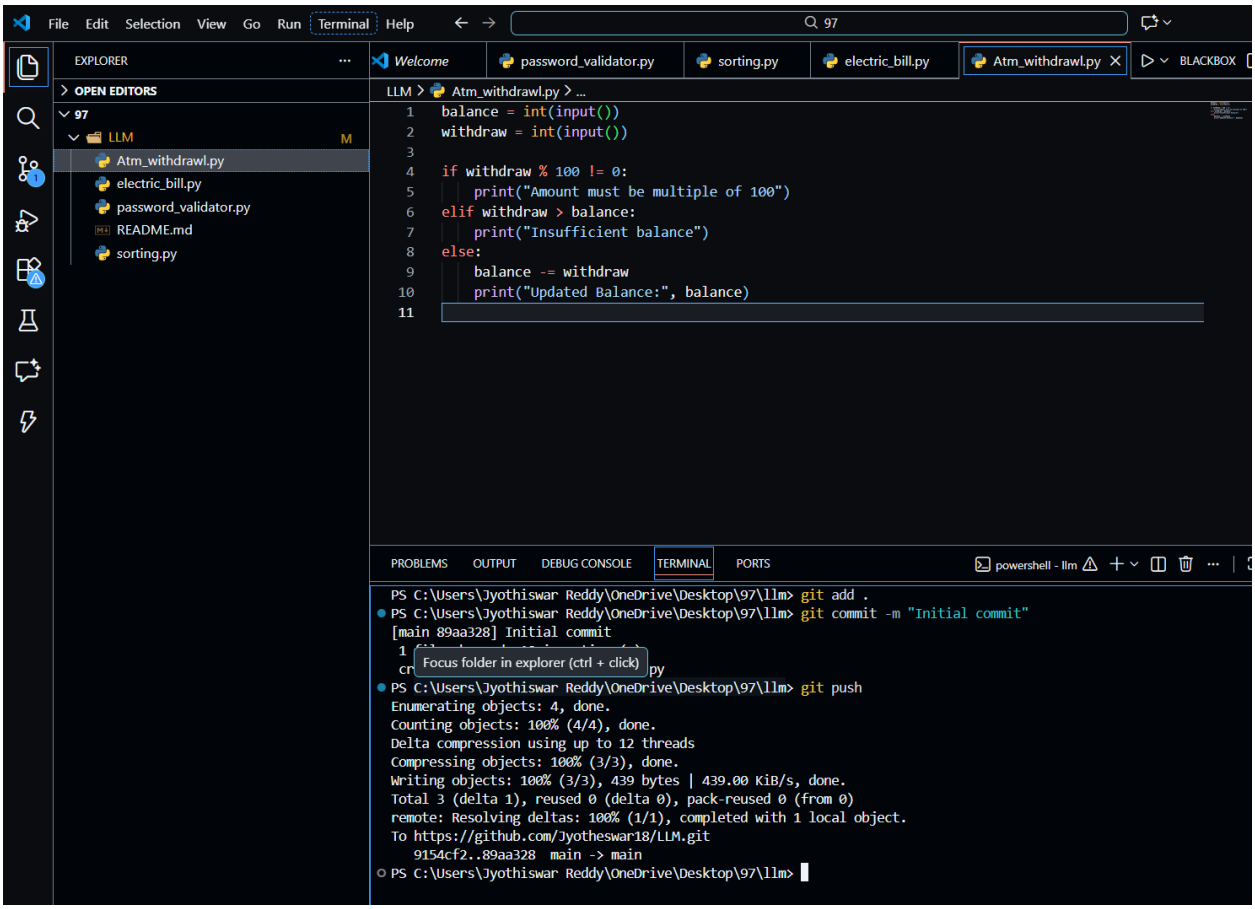


4) An ATM machine allows withdrawals only if sufficient balance is available.
Write a program that: Stores a user's balance

•Takes withdrawal amount as input

• Checks: Amount must be a multiple of 100 o Balance must be sufficient

• Deducts amount and displays updated balance



5) A company calculates monthly salary based on basic pay. Write a program that:

• Accepts basic salary

• Calculates: HRA = 20% of basic

DA = 10% of basic

Tax = 5% of total salary

• Displays net salary

File Edit Selection View Go Run Terminal Help

EXPLORER

OPEN EDITORS

97

LLM M

- Atm_withdrawl.py
- electric_bill.py
- password_validator.py
- README.md
- salary.py
- sorting.py

password_validator.py | sorting.py | electric_bill.py | Atm_withdrawl.py | salary.py

LLM > salary.py > ...

```python
basic = float(input())
hra = 0.2 * basic
da = 0.1 * basic
total = basic + hra + da
tax = 0.05 * total
net_salary = total - tax
print("Net Salary:", net_salary)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS powershell - llm

```
PS C:\Users\Jyothiswar Reddy\OneDrive\Desktop\97\llm> git add .
PS C:\Users\Jyothiswar Reddy\OneDrive\Desktop\97\llm> git commit -m "Initial commit"
[main 3bc9224] Initial commit
 1 file changed, 7 insertions(+)
 create mode 100644 salary.py
PS C:\Users\Jyothiswar Reddy\OneDrive\Desktop\97\llm> git push
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 12 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 399 bytes | 399.00 KiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/Jyotheswar18/LLM.git
   89aa328..3bc9224  main -> main
PS C:\Users\Jyothiswar Reddy\OneDrive\Desktop\97\llm>
```